

---

# Attention Is All You Need

---

Ashish Vaswani\*  
Google Brain  
avaswani@google.com

Noam Shazeer\*  
Google Brain  
noam@google.com

Niki Parmar\*  
Google Research  
nikip@google.com

Jakob Uszkoreit\*  
Google Research  
usz@google.com

Llion Jones\*  
Google Research  
llion@google.com

Aidan N. Gomez\*<sup>†</sup>  
University of Toronto  
aidan@cs.toronto.edu

Łukasz Kaiser\*  
Google Brain  
lukaszkaizer@google.com

Illia Polosukhin\*<sup>‡</sup>  
illia.polosukhin@gmail.com

## AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy\*,<sup>†</sup>, Lucas Beyer\*, Alexander Kolesnikov\*, Dirk Weissenborn\*,  
Xiaohua Zhai\*, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,  
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby\*,<sup>†</sup>

\*equal technical contribution, <sup>†</sup>equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

## Scalable Diffusion Models with Transformers

William Peebles\*  
UC Berkeley

Saining Xie  
New York University

Transformer



ViT(Vision Transformer)



DiT(diffusion Transformer)

张雨阳  
2026.6.25

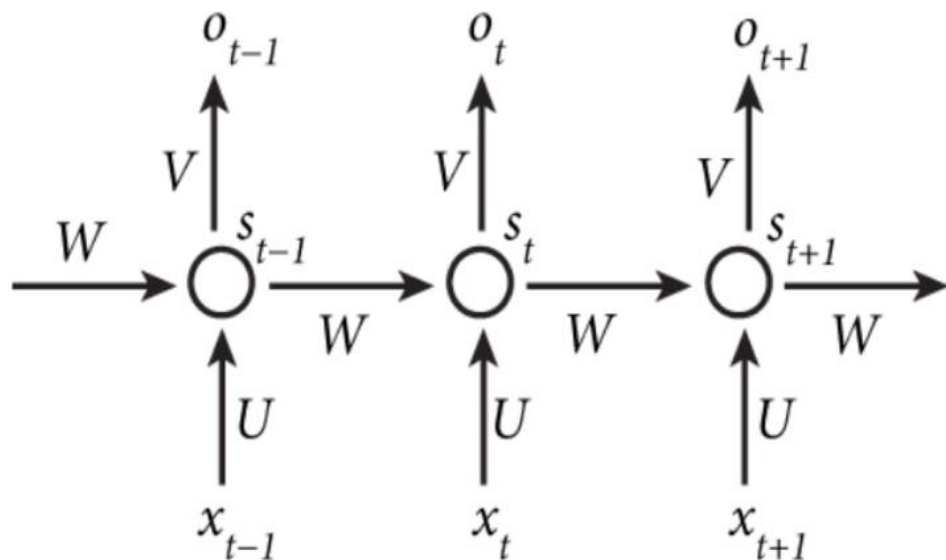
# 汇报主线：先讲架构，再统一比较

---

- 1 Transformer 把序列中的 token 通过 self-attention 进行全局交互
- 2 ViT 把图像切成 patch, 把 patch 当作 token 输入 Transformer Encoder
- 3 DiT 把扩散模型的 U-Net backbone 换成处理 latent patch 的 Transformer
- 4 实验对比 分别看翻译、分类、生成任务中的关键证据
- 5 统一理解 token 化 + Transformer backbone + task head/decoder 的演化路径

# Transformer

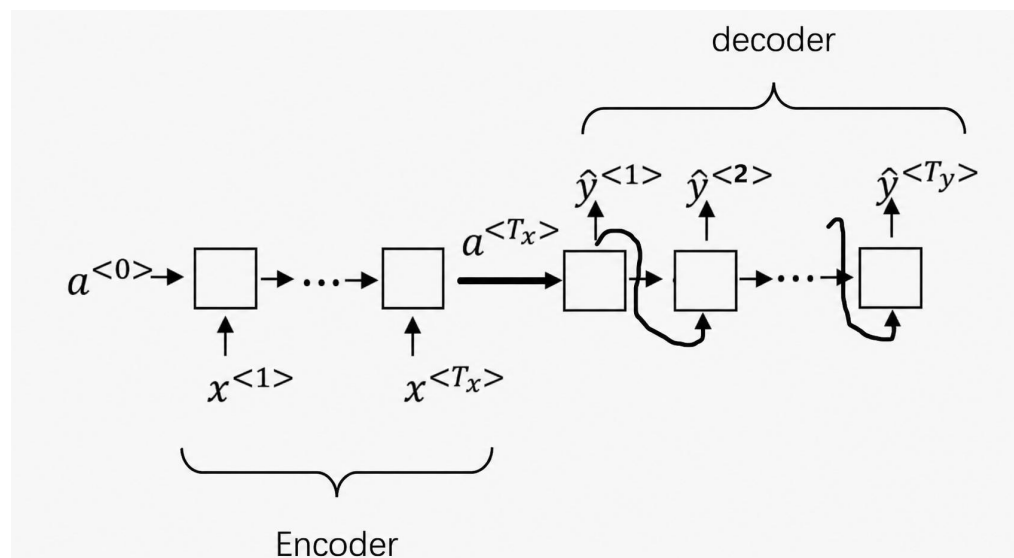
传统的RNN架构



不足:

1. 无法并行计算
2. 输入和输出等长

Encoder-Decoder架构

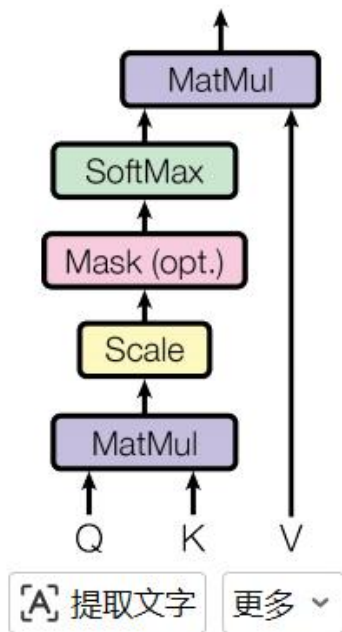


编码器和解码器之间只通过一个隐状态来传递信息。在处理较长的文章时，这种架构的表现不够理想。

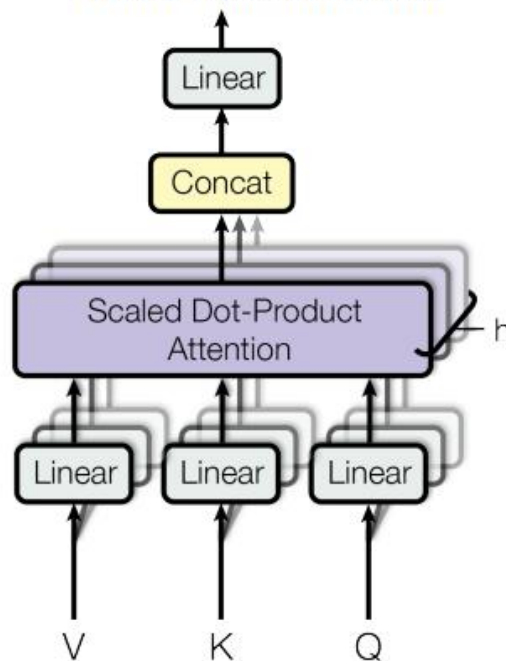
# Transformer

Transformer 的目标: 完全依赖 attention, 去掉 recurrence 和 convolution

Scaled Dot-Product Attention



Multi-Head Attention



$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

$$W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \\ \text{where } \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

# Transformer

## Encoder

- N=6 层，每层包含 Multi-Head Self-Attention 与 FFN。
- 每个子层外都有 residual connection + layer normalization。
- 输入 token embedding 加 positional encoding。

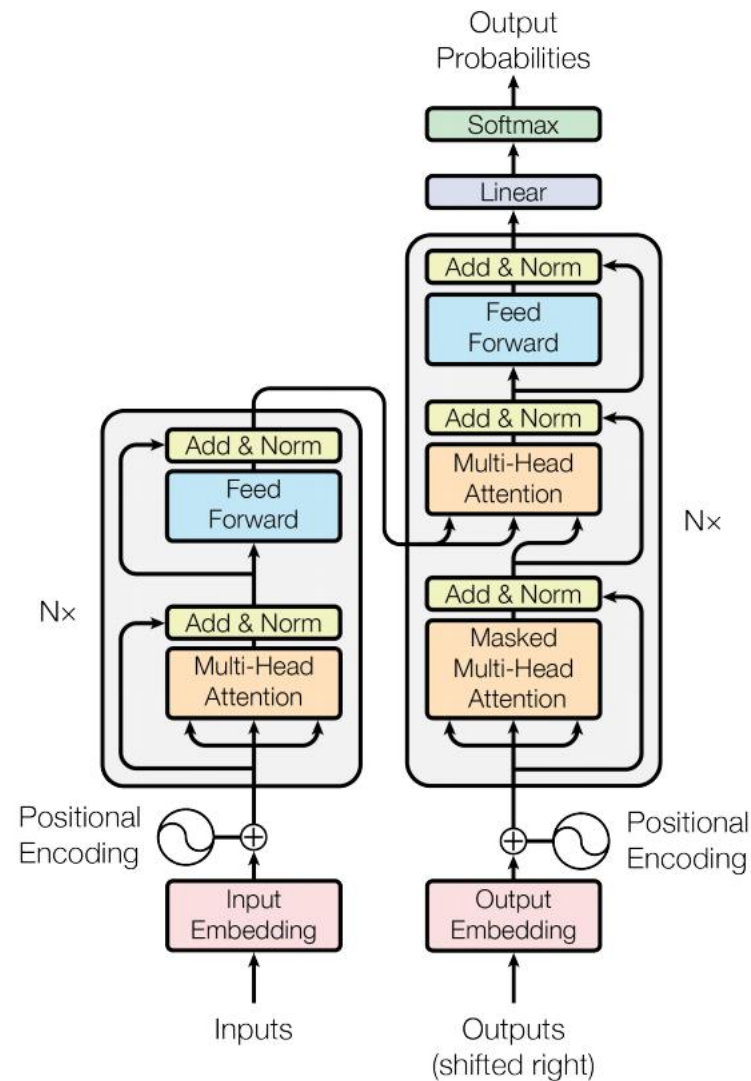
### Position-wise Feed-Forward Networks

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

mask机制:  $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right)V$

## Decoder

- Masked self-attention 保证自回归生成。
- 通过 encoder-decoder attention 读取源序列。
- 最后接 Linear + Softmax 输出 token 概率。



# Transformer

---

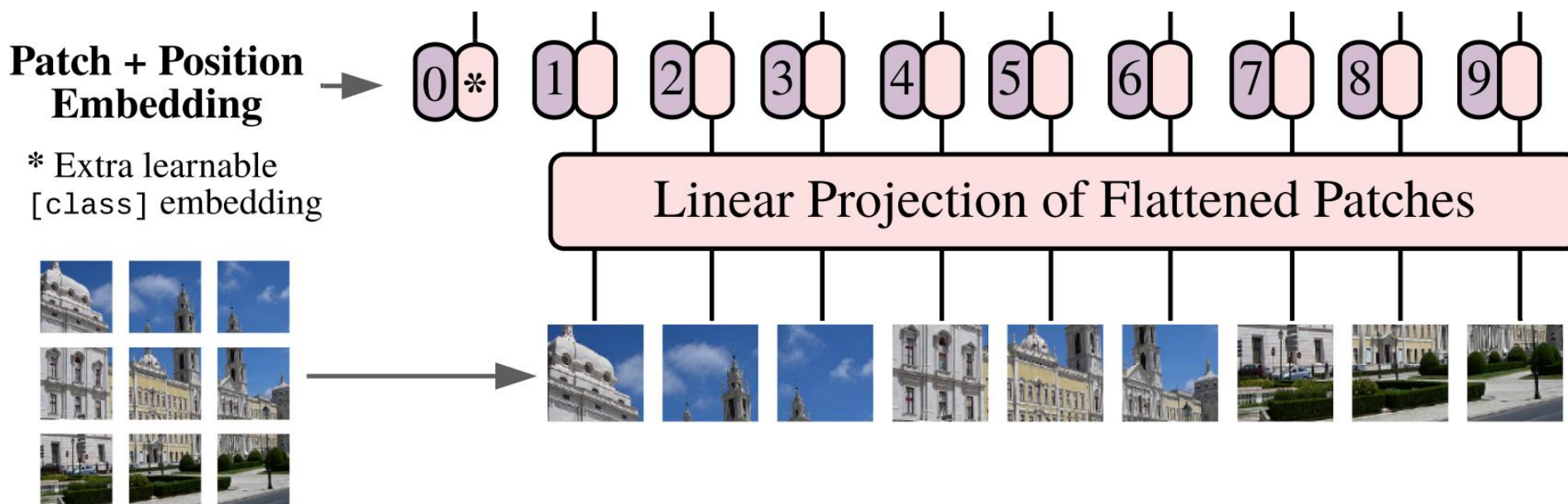
我们可以从transformer中抽象出一个通用模板



- 这三篇文章的共同点不是任务相同，而是把不同数据类型都变成“token 序列”。
- 后续的变化主要集中在：token 怎么构造、如何注入位置信息、输出头如何适配任务。

# ViT: 把图像识别问题改写成序列建模

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \cdots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$$



$$\mathbf{x} \in \mathbb{R}^{H \times W \times C}$$

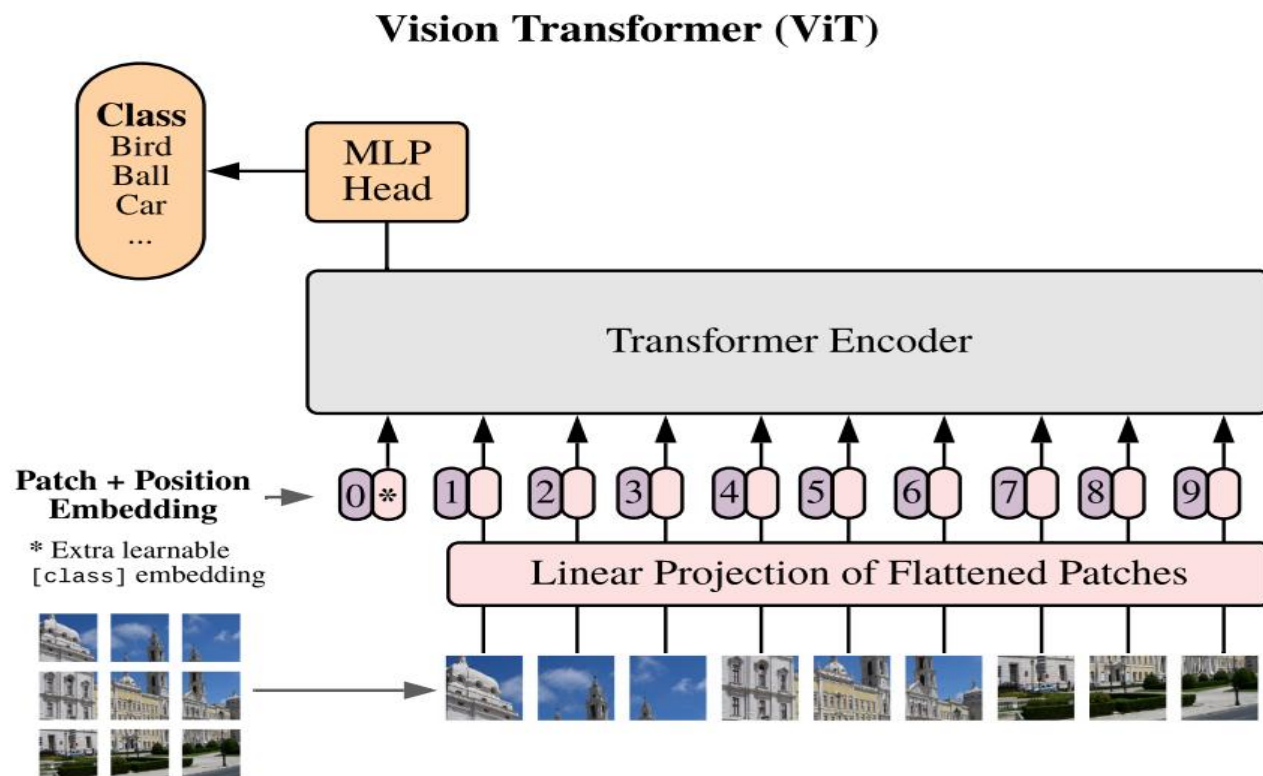
$$\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$$

$$N = HW / P^2$$

(P, P) 是每个图像块的分辨率

(H, W) 是原图分辨率

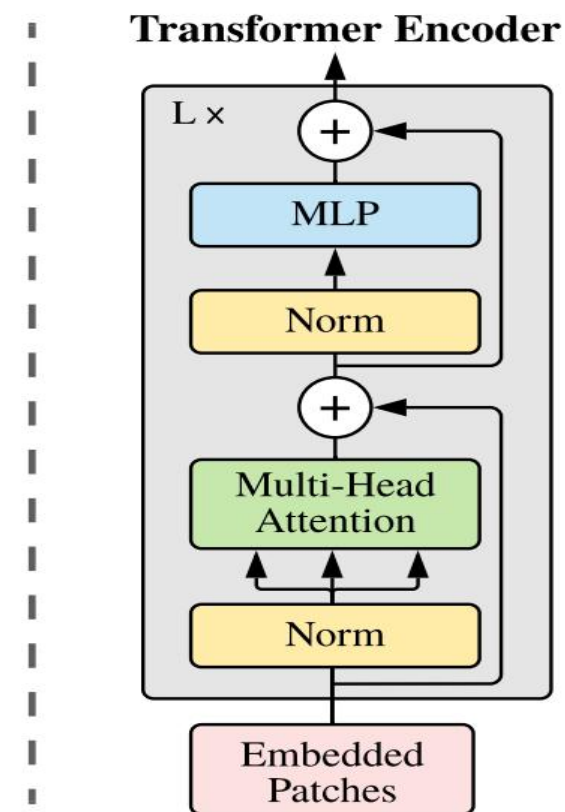
# ViT: 把图像识别问题改写成序列建模



$$\mathbf{z}'_{\ell} = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1},$$

$$\mathbf{z}_{\ell} = \text{MLP}(\text{LN}(\mathbf{z}'_{\ell})) + \mathbf{z}'_{\ell},$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0)$$



$$\ell = 1 \dots L$$

$$\ell = 1 \dots L$$

# ViT: 把图像识别问题改写成序列建模

---

baked into each layer throughout the whole model. In ViT, only MLP layers are local and translationally equivariant, while the self-attention layers are global. The two-dimensional neighborhood

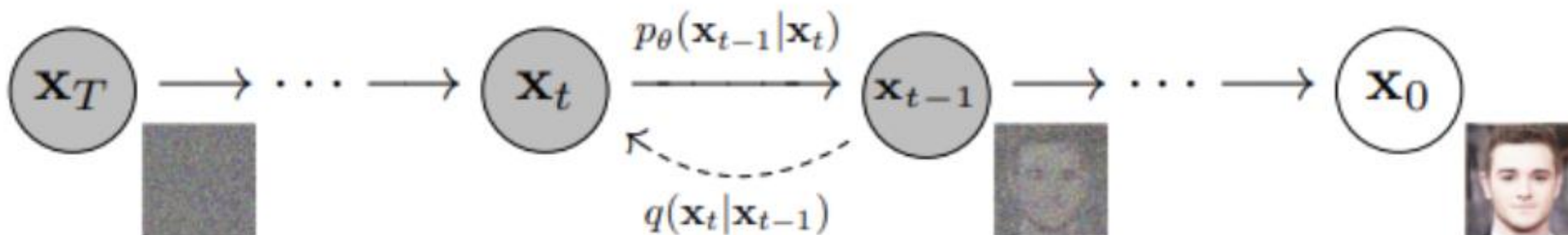
CNN:

- 局部性 (Locality): 假设图像中相邻的像素之间相关性最强 (卷积核只在局部感受野内操作)。
- 二维邻域结构 (Two-dimensional neighborhood structure): 网络结构天然理解上下左右的空间关系 (二维卷积、池化)。
- 平移等变性 (Translation equivariance): 无论目标出现在图像的左上角还是右下角, 同样的卷积核提取出的特征是相同的 (共享权重)。

ViT 的关键假设: 足够大数据下, 模型可以自己学到图像结构

## DiT: 把扩散模型的 U-Net backbone 换成 Transformer

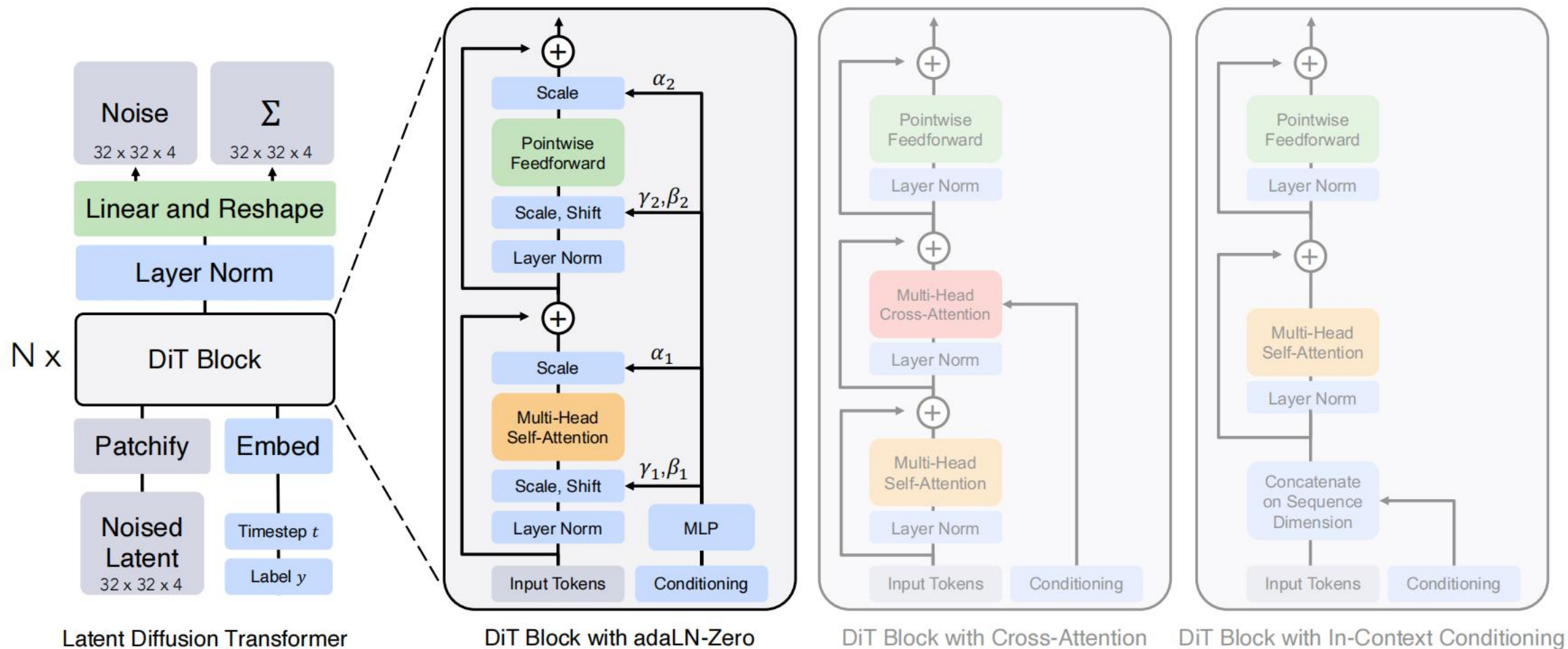
### Scalable Diffusion Models with Transformers



- 扩散模型通常使用卷积 U-Net 作为 denoising backbone。
- U-Net 的归纳偏置并非必须，能用更通用的 Transformer。
- 做法：在 latent diffusion 框架下，对 VAE latent patchify，再用 Transformer 预测噪声和方差。

# DiT 架构: latent patch 输入 + 条件注入 + 线性解码

$$(z_t, t, y) \longrightarrow (\epsilon_\theta(z_t, t, y), \Sigma_\theta(z_t, t, y))$$



## Just Image Transformers(JiT)

图片分块的Transformer

$$H \times W \times C \text{ 图像} \longrightarrow \frac{H}{p} \times \frac{W}{p} \times C \text{ patch}$$

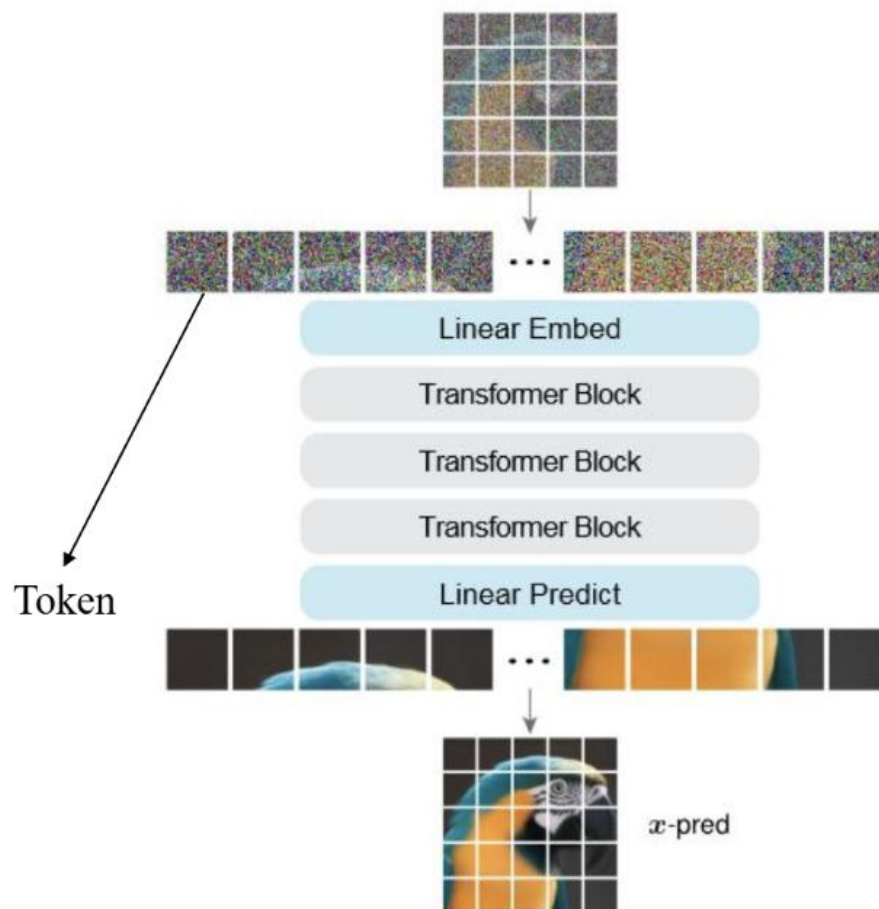
极简主义的设计：无分类器、预训练、额外损失

但 DiT 仍然依赖**两层外部设计**：

$$x \xrightarrow{\text{VAE Encoder}} z \xrightarrow{\text{DiT diffusion}} \hat{z} \xrightarrow{\text{VAE Decoder}} \hat{x}$$

JiT 则进一步**简化**为：

$$x_t \xrightarrow{\text{Pixel-space Transformer}} \hat{x}_0$$



# 实验 1: Transformer 在机器翻译上验证架构有效性

| 模型               | EN-DE BLEU | EN-FR BLEU | 训练成本                  |
|------------------|------------|------------|-----------------------|
| ConvS2S Ensemble | 26.36      | 41.29      | 7.7e19 / 1.2e21 FLOPs |
| Transformer Base | 27.3       | 38.1       | 3.3e18 FLOPs          |
| Transformer Big  | 28.4       | 41.8       | 2.3e19 FLOPs          |

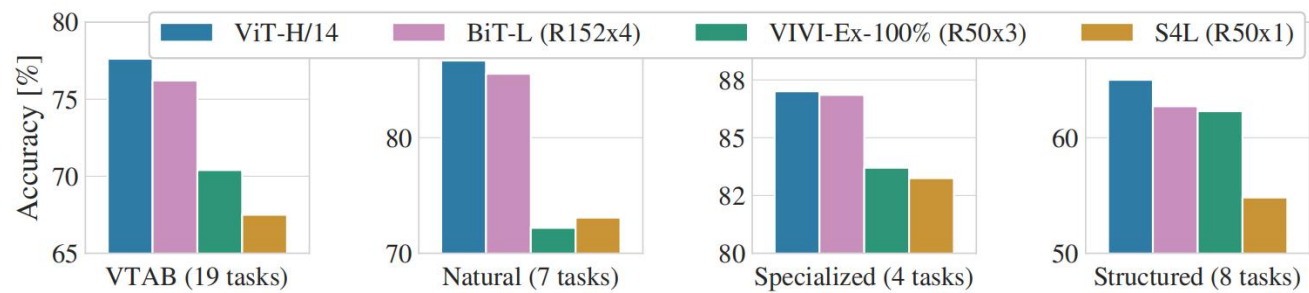
- WMT14 English-German: Transformer Big 达到 28.4 BLEU，超过之前集成模型。
- WMT14 English-French: 表中达到 41.8 BLEU。
- 训练设置: Base 约 12 小时; Big 约 3.5 天, 8 张 P100 GPU。

结论: 更高 BLEU + 更高并行度 + 更低训练成本

# 实验 2: ViT 的关键不是小数据, 而是大规模预训练

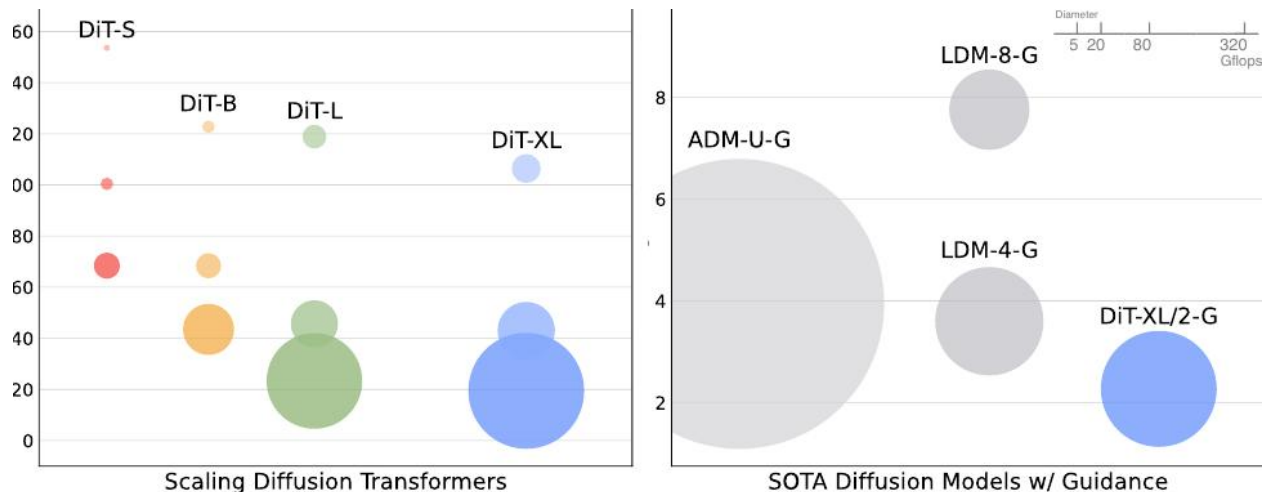
|                    | Ours-JFT<br>(ViT-H/14)  | Ours-JFT<br>(ViT-L/16)  | Ours-I21k<br>(ViT-L/16) | BiT-L<br>(ResNet152x4) | Noisy Student<br>(EfficientNet-L2) |
|--------------------|-------------------------|-------------------------|-------------------------|------------------------|------------------------------------|
| ImageNet           | <b>88.55</b> $\pm 0.04$ | 87.76 $\pm 0.03$        | 85.30 $\pm 0.02$        | 87.54 $\pm 0.02$       | 88.4/88.5*                         |
| ImageNet ReaL      | <b>90.72</b> $\pm 0.05$ | 90.54 $\pm 0.03$        | 88.62 $\pm 0.05$        | 90.54                  | 90.55                              |
| CIFAR-10           | <b>99.50</b> $\pm 0.06$ | 99.42 $\pm 0.03$        | 99.15 $\pm 0.03$        | 99.37 $\pm 0.06$       | —                                  |
| CIFAR-100          | <b>94.55</b> $\pm 0.04$ | 93.90 $\pm 0.05$        | 93.25 $\pm 0.05$        | 93.51 $\pm 0.08$       | —                                  |
| Oxford-IIIT Pets   | <b>97.56</b> $\pm 0.03$ | 97.32 $\pm 0.11$        | 94.67 $\pm 0.15$        | 96.62 $\pm 0.23$       | —                                  |
| Oxford Flowers-102 | 99.68 $\pm 0.02$        | <b>99.74</b> $\pm 0.00$ | 99.61 $\pm 0.02$        | 99.63 $\pm 0.03$       | —                                  |
| VTAB (19 tasks)    | <b>77.63</b> $\pm 0.23$ | 76.28 $\pm 0.46$        | 72.72 $\pm 0.21$        | 76.29 $\pm 1.70$       | —                                  |
| TPUv3-core-days    | 2.5k                    | 0.68k                   | 0.23k                   | 9.9k                   | 12.3k                              |

- JFT-300M 预训练后, ViT-H/14 在 ImageNet 达到 88.55%, CIFAR-100 达到 94.55%, VTAB 达到 77.63。
- 与 BiT-L 相比, ViT-H/14 使用更少 TPUv3-core-days, 同时多个 benchmark 更优。
- 数据规模很关键: ImageNet 小数据时 ResNet 更稳; ImageNet-21k/JFT 规模下 ViT 才显现优势。



结论: 大规模训练可以“弥补”视觉归纳偏置不足

## 实验 3: DiT 证明 Transformer backbone 可扩展到生成任务



- FID 随 DiT 模型计算量增加而稳定下降: Gflops 与 FID 呈强负相关。
- 更大的模型更 compute-efficient, 小模型训练更久也难以追上大模型。
- ImageNet  $256 \times 256$ : DiT-XL/2 + classifier-free guidance 将 FID-50K 降至 2.27。
- ImageNet  $512 \times 512$ : DiT-XL/2-G 达到 3.04 FID, 超过此前最佳扩散模型。

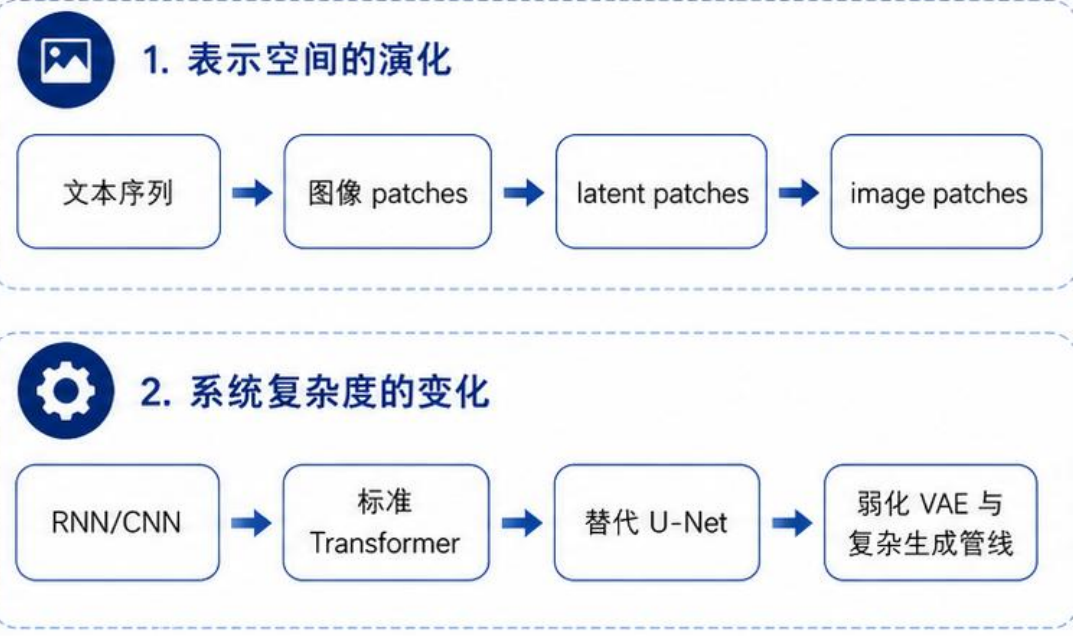
结论: 扩散模型也可以享受 Transformer scaling

# 演化路径与核心结论

## 演化路径



## 两条主线



### 一句话总结

从 Transformer 到 ViT、DiT、JiT、本质上是在不断验证：只要把输入变成合适的 token，并配合足够的数据与计算，Transformer 就能成为跨任务的统一 backbone。



但不同任务仍然需要不同的输入表示、条件设计和训练目标。

统一主线：不同数据 → token 化 → Transformer 全局交互 → 任务输出



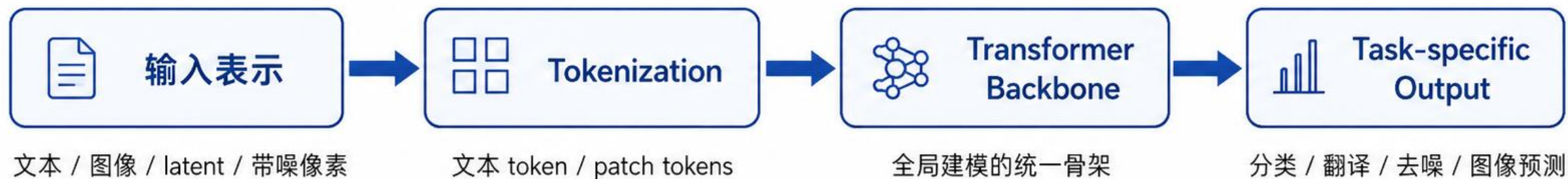
# 四个模型的统一对比（总览）

|             | Transformer                 | ViT                 | DiT                                   | JiT               |
|-------------|-----------------------------|---------------------|---------------------------------------|-------------------|
| 核心任务        | 序列建模 / 机器翻译                 | 图像分类                | 扩散生成 / 图像生成                           | 图像生成 / 图像预测       |
| 输入表示        | 文本序列                        | 图像                  | 带噪 latent                             | 带噪像素              |
| Token 化方式   | 文本 token                    | 图像切成 Patch Tokens   | latent 切成 Patch Tokens                | image patches     |
| Backbone 结构 | Encoder-Decoder Transformer | Transformer Encoder | 条件 Transformer Denoiser               | Image Transformer |
| 输出目标        | 下一个 token / 翻译序列            | 图像类别                | 噪声 / 协方差预测                            | 干净图像 / x0 预测      |
| 替代了什么       | RNN / 序列 CNN                | CNN 分类 backbone     | 扩散模型中的 U-Net                          | 进一步弱化 VAE 与复杂生成管线 |
| 核心优势        | 全局依赖、并行训练、通用性强              | 架构简洁、可扩展、适合大规模预训练   | scaling 好、生成质量高、<br>统一 Transformer 骨架 | 结构更简洁、直接在图像空间建模   |
| 主要局限        | 推理仍具自回归顺序性                  | 小数据下依赖不足，归纳偏置较弱     | 仍依赖 VAE latent，计算量较高                  | 计算与训练稳定性仍待进一步验证   |



**统一视角：** 输入表示不同，但都遵循 Tokenization → Transformer Backbone → Task-specific Output

# 统一理解：其实都在做同一件事



## 1. Transformer

- 文本序列 → 文本 token
- Backbone: Encoder-Decoder
- 输出: 翻译序列 / 下一个 token

## 2. ViT

- 图像 → Patch Tokens
- Backbone: Transformer Encoder
- 输出: 图像类别

## 3. DiT

- 带噪 latent → Latent Patch Tokens
- Backbone: 条件 Transformer Denoiser
- 输出: 噪声 /  $x_0$  相关预测

## 4. JiT

- 带噪 Pixels → Image Patch Tokens
- Backbone: Image Transformer
- 输出: 干净图像预测



**统一理解：** 差别主要不在 Transformer block 本身，而在 **输入表示、条件注入方式** 与 **输出目标**。